

Process Model Selection and Project Plan "Git Visualiser"

Aim

To develop a project plan for the "Git Visualiser" using the Agile software process model, outline its phases, key activities, deliverables, and analyze its advantages and challenges.

Problem Statement

"Git Visualiser" is a web-based dashboard application that helps developers understand the evolution of a Git repository at a glance. The system visualizes four core aspects:

- **Commit History:** A chronological, searchable timeline of commits with metadata.
- **Branch & Merge Structure:** An interactive graph showing branch divergence and merging.
- **Contributor Analytics:** Statistics on individual contributor activity over time.
- **File Change Patterns:** Frequency of file changes and insertion/deletion trends.

The application extracts metadata by connecting to a local repository or a remote host (like GitHub) and presents it through an interactive dashboard.

PART I: Analysis & Selection

1. Nature of the Project

- **Size:** Small-to-medium, realistically buildable by a small team of 2-3 students within a single college semester.
- **Complexity:** Moderate; the primary challenge lies in parsing Git's internal object models and transforming that data into interactive visualizations.
- **User Involvement:** Moderate; continuous feedback will come from self-testing, peer review, and faculty evaluation at fixed lab checkpoints.
- **Time Constraints:** Fixed by an academic semester schedule featuring intermediate deadlines rather than one distant delivery date.
- **Requirement Clarity:** High for core features, but exact UI/UX execution and the technology stack are expected to be refined as development progresses.

2. Process Model Comparative Analysis

The following table provides a descriptive evaluation of models based on the specific constraints of the Git Visualiser project:

Evaluation Criteria	Agile (Selected)	Waterfall	Incremental	Spiral
Project Size (Small-to-medium)	Highly Suitable: Small scope maps perfectly to manageable sprints.	Suitable	Suitable	Overkill for scope.
Complexity (Moderate, tech-centric)	Suitable: Allows for technical exploration (spikes) to handle data parsing.	Unsuitable: Too rigid if technical hurdles arise.	Suitable	Suitable
Time to Market (Fixed academic checkpoints)	Highly Suitable: Delivers working components exactly in time for lab evaluations.	Unsuitable: Only delivers at the very end.	Suitable	Unsuitable
User Interaction (Moderate feedback loops)	Highly Suitable: Adapts easily to peer and faculty feedback during reviews.	Unsuitable: Locks out feedback until completion.	Moderate	Moderate
Requirement Flexibility (Evolving UI/UX)	Highly Suitable: Built to embrace evolving visual polish and tech stack changes.	Unsuitable	Low	High

3. Selected Process Model & Justification

The **Agile Model** is selected as the most suitable process for developing Git Visualiser.

- **Why Agile?** The project's four core features divide naturally into independent development sprints. Agile values early, working software, enabling the team to continuously integrate and present functional dashboard views at every academic checkpoint. It perfectly accommodates the expected refinement of UI/UX details as the team receives ongoing faculty feedback.
- **Compared to Incremental:** While Incremental also delivers early software, Agile is specifically optimized for iterative refinement. Instead of locking in the design for each module upfront, Agile allows the visualizations to evolve organically based on what is learned in previous sprints.
- **Compared to Waterfall:** Waterfall is rejected because it requires freezing all requirements upfront and only produces a single delivery at the end. If any phase overruns, the team would have zero working software to demonstrate during intermediate lab evaluations.

PART II: Execution Plan

1. Agile Sprints for Git Visualiser

The project will be executed in four primary Sprints, with each sprint cycling through Planning, Design, Development, Testing, and Deployment.

- **Sprint 1:** Core Repository Parsing & Commit History View.
- **Sprint 2:** Branch & Merge Graph Visualization.
- **Sprint 3:** Contributor Analytics.
- **Sprint 4:** File Change Tracking & Dashboard Integration.

2. Activities and Deliverables per Sprint

Sprint	Key Activities	Sprint Deliverable
1. Commit History	Determine required commit fields. Wireframe timeline UI. Build parsing module. Verify data against raw git logs.	Working commit history viewer.
2. Branch Graph	Design visual representation of branches and nodes. Build extraction logic. Render interactive graph.	Working branch/merge visualization.
3. Analytics	Define key metrics (LOC, recency). Develop charting components. Aggregate author data and cross-check.	Working contributor analytics panel.
4. Integration	Map file-level metrics. Design unified dashboard layout. Conduct end-to-end usability testing.	Fully integrated Git Visualiser.

3. Advantages and Limitations of Agile for This Project

Advantages:

- Functional software is available after every sprint, allowing for demonstrable progress at each lab checkpoint.
- Lessons learned regarding visualization libraries or parsing logic in early sprints can immediately improve subsequent sprints.
- Even if the final sprint (file tracking) is delayed, the earlier deliverables still form a highly usable product.

Limitations:

- Requires continuous testing and integration overhead to ensure new sprints do not break older features.
- Agile carries a risk of scope creep; the team might endlessly refine early UI features instead of moving on to core back-end parsing.

- The overall data architecture must still be carefully planned early on to avoid severe refactoring when integrating complex file and contributor data later.

FAQs

1. If two projects have the same budget but different requirements, would you choose the same process model?

No. Budget indicates spending capacity, but the stability and clarity of the requirements drive process model selection. A project with well-defined requirements suits Waterfall or Incremental, while a project with volatile requirements demands the flexibility of Agile, regardless of funding.

2. Why is software testing integrated into every process model?

Every model must ultimately deliver functional software, and defects are drastically cheaper to fix the earlier they are found. A bug caught during an early sprint is inexpensive to resolve, whereas the same bug found in production is highly disruptive and costly.

3. What factors should be considered while selecting a software process model?

Key factors include requirement clarity and stability, project size and complexity, timeline flexibility, team experience, risk tolerance, budget, and the level of available user involvement.